

Es hora de que pongas en práctica todo lo aprendido. 

En este ejercicio vamos a crear un proyecto Spring que deberá tener Clases Entidad, Repositorio y Servicio.

Más adelante conseguirás las resoluciones para que valides tus respuestas y puedas monitorear tu progreso. 

¡Manos a la obra!

1. Desafío

Crear un proyecto Spring con las dependencias: **Spring Web, Spring Data y Spring Data JPA**. Para ello, será necesario generar las siguientes clases:

- a. **Clase Persona:** Esta clase entidad modelará objetos que deben tener las siguientes características:
 - Integer **rut** (que será el identificador),
 - String **nombre**,
 - Integer **edadm**
 - Mascota **mascota**, que será un atributo de tipo Mascota, por lo cuál deberás mapear debidamente la relación (para este ejercicio, vamos a determinar que una Persona solo puede tener una Mascota).
 - Además, se deberán generar los constructores y métodos modificadores y accesadores necesarios.
- b. **Clase Mascota:** Esta clase entidad modelará objetos que deben tener las siguientes características:
 - Integer **Id**,
 - String **Apodo**,
 - String **Raza**.
 - Además, se deberán generar los constructores y métodos modificadores y accesadores necesarios.

- c. Las interfaces **PersonaRepositorio** y **MascotaRepositorio**. serán las encargadas de interactuar con la capa de acceso a datos. Debes generar sólo las estructuras sin implementar ningún método.
- d. Una vez generados estos archivos, debes crear las clases **PersonaServicio** y **MascotaServicio**.
 - En la clase **MascotaServicio** debe existir un método para guardar mascotas en el repositorio. Por ejemplo: método Mascota guardar (Mascota mascota) = debería guardar la mascota que recibe por parámetro..
 - En la clase **PersonaServicio** debe existir un método para listar todos los objetos de la clase Persona. Por ejemplo: método Persona listar() = debería retornar todas las personas que existen.
 - Es necesario declarar los atributos, variables y métodos con las anotaciones pertinentes:
 - @Entity - @Service - @Repository
 - @Id
 - @GeneratedValue
 - @"Relacion"
 - @Autowired
 - @Transactional

2. ¿Dónde se lleva a cabo? 🏠

Eclipse IDE o Spring Tool Suite

3. Tiempo de dedicación ⌚

1 hora

4. ⚠ Condición

Esta práctica o ejercitación **no requiere ser entregada y/o evaluada** por el mentor. No obstante puedes compartir tus resultados con el resto de los bootcampers y construir conocimiento en conjunto.

Resolución del desafío 📌

Respetando el pedido del desafío, las clases Persona y Mascota quedarían codificada de la siguiente manera:

```
Persona.java X
1 package com.ejercicio.spring.entidades;
2
3 import jakarta.persistence.Entity;
9 @Entity
10 public class Persona {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Integer rut;
15     private String nombre;
16     private Integer edad;
17     @OneToOne
18     private Mascota mascota;
19
20     // constructores
21     public Persona() {}
24
25     public Persona(int rut, String nombre, int edad, Mascota mascota) {}
31
32     // Gets & Sets
33     public int getRut() {}
36
37     public void setRut(int rut) {}
40
41     public String getNombre() {}
44
45     public void setNombre(String nombre) {}
48
49     public int getEdad() {}
52
53     public void setEdad(int edad) {}
56
57     public Mascota getMascota() {}
60
61     public void setMascota(Mascota mascota) {
62         this.mascota = mascota;
63     }
64
65 }
```

```

1 Mascota.java X
1 package com.ejercicio.spring.entidades;
2 import jakarta.persistence.Entity;
6
7 @Entity
8 public class Mascota {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Integer id;
13     private String apodo;
14     private String raza;
15
16     public Mascota() {}
19
20     public Mascota(int id, String apodo, String raza) {}
25
26     public int getId() {}
29
30     public void setId(int id) {}
33
34     public String getApodo() {}
37
38     public void setApodo(String apodo) {}
41
42     public String getRaza() {}
45
46     public void setRaza(String raza) {}
49
50 }

```

Por otro lado, las interfaces **PersonaRepositorio** y **MascotaRepositorio** sólo debían crearse, pero no declarar métodos ni implementarlos:

```

1 *PersonaRepositorio.java X
1 package com.ejercicio.spring.repositorios;
2 import org.springframework.data.repository.CrudRepository;
6
7 @Repository
8 public interface PersonaRepositorio extends CrudRepository<Persona,Integer > {
9
10 }

```

```

1 MascotaRepositorio.java X
1 package com.ejercicio.spring.repositorios;
2
3 import org.springframework.data.repository.CrudRepository;
7
8 @Repository
9 public interface MascotaRepositorio extends CrudRepository<Mascota, Integer>{
10
11
12 }

```

Las clases que se encargan de la lógica del negocio quedarían codificadas de la siguiente manera, declarando un método por clase como pide el enunciado.

```
PersonaServicio.java X
1 package com.ejercicio.spring.servicios;
2
3+ import java.util.List;[]
11
12 @Service
13 public class PersonaServicio {
14
15-   @Autowired
16   private PersonaRepositorio personaRepositorio;
17
18
19-   @Transactional(readonly = true)
20   public List<Persona> listar(){
21
22       return (List<Persona>) personaRepositorio.findAll();
23   }
24
25 }
```

```
MascotaServicio.java X
1 package com.ejercicio.spring.servicios;
2
3+ import org.springframework.beans.factory.annotation.Autowired;[]
9
10 @Service
11 public class MascotaServicio {
12
13-   @Autowired
14   private MascotaRepositorio mascotaRepositorio;
15
16-   @Transactional
17   public Mascota guardar (Mascota mascota) {
18
19       return mascotaRepositorio.save(mascota);
20   }
21 }
22 }
```

Por último, veamos el árbol del proyecto. Esta división en capas ayuda a mantener el código organizado. Es necesario que cualquier persona que intente leer nuestro código, pueda acceder fácilmente a las clases que necesite. Es por eso que se recomienda el uso de palabras descriptivas.

